Applied Software Development Lab

Lab Handout 5: The word `class` makes an appearance

Learning objectives

- **Reading the serial interface with Python**
- **Using control structures (`if`, `for`, `while`) and simple data types**
- **A step-by-step introduction to object-oriented programming (including the motivation, but without overkill – the rest comes later)**

Step 1: Cloning your own repository and building the game controller

Goal: Picking up where we left off last time – starting with the microcontroller code.

On the breadboard, rebuild the game controller from Lab Handout 4, connect it to the microcontroller, and connect the microcontroller to your computer via USB.

As always:
- Open two terminals (left for editing, right for running commands).
- In addition, open a web browser and go to your Git repository at `https://gitlab.uni-ulm.de/USERNAME/asp-ulm`

Left terminal (editing)

A) **Clone the repository and change the directory**
   Clone the repository with:
   `git clone https://gitlab.uni-ulm.de/USERNAME/asp-ulm`
   Change the directory:
   `cd asp-ulm/controller`

B) **Open the editor**
   Open the file `controller.ino` in the editor of your choice.

C) **Think ;-)**
   Read the code carefully and think about what should happen when you press the push button or turn the potentiometer.

Right terminal (running commands)

A) **Change the directory**
   `cd asp-ulm/controller`

B) **Make the USB port visible to WSL**
   `usb_check`

C) **Flash the microcontroller**
   `make upload`

D) **Start the serial monitor**
   `make monitor`

E) **Test**
   Check whether the expected behavior occurs when you press the push button and turn the potentiometer.

Step 2: A serial monitor of our own making

Goal: Reading the serial interface with a Python program.

What is special about our own version: the output is not produced character by character, but only once a complete line (that is, up to the line break) has been received. As soon as that is the case, **this line is output all at once**. As long as that is not the case, nothing happens.

Preparation

1. **Left terminal**
   A) **Change the directory**
   `cd ../python`
   B) **Open the editor**
   Open the file `serial-reader.py` in the editor of your choice.
   C) **Think ;-)**
   Read the code carefully and think about what should happen when you press the push button or turn the potentiometer. If necessary, go through Step 6 of Lab Handout 4 again to make sure you understand:
   (i) What do `ser.in_waiting` and `ser.read` do?
   (ii) What do the remaining statements do?
   (iii) How are the loops nested – and why?
2. **Right terminal**
   A) Change the directory
   `cd ../python`
   B) Test
   Start the program and check whether the expected behavior occurs when you press the push button and turn the potentiometer.

You should now be at the point where individual bytes are received. For example, `ch` is output as `b'X'` – an object of type bytes, i.e., a data type for "raw" binary data. How to convert `ch` into readable text is something we will explore experimentally in a Python shell in a moment.

Tasks

1. If you have not already done so, stop the Python program in the right terminal with `Control-C`.

2. Start the interactive Python shell with `python`
3. The Python shell works much like the terminal shell you use to navigate the file system or run commands – except that here you type in Python statements.

   Enter the following, one after the other:
   (1) `ch = b'A'`
   (2) `print(ch)`
   (3) `type(ch)`
   (4) `s = bytes.decode(ch)`
   (5) `type(s)`

© Michael Lehn | CC BY-SA 4.0

You should observe:

```
>>> ch = b'A'
>>> print(ch)
b'A'
>>> type(ch)
<class 'bytes'>
>>> type(ch)
<class 'bytes'>
>>> s = bytes.decode(ch)
>>> type(s)
<class 'str'>
```

You can use `type(...)` to query the type of a variable. And the function `bytes.decode(...)` lets you convert a bytes object such as `ch` into a string.

4. **Tip for using the Python shell:** With the **up and down arrow keys** you can scroll through the input history and run or edit commands you have already entered.

5. Now we explore experimentally how strings can be concatenated. To do this, enter the following in the Python shell:
   (1) `buffer = ""`
   (2) `print(buffer)`
   (3) `buffer = buffer + s`
   (4) `print(buffer)`
   (5) `buffer += s`
   (6) `print(buffer)`

   You see:
   - With `+` you can concatenate two strings.
   - The notation `buffer += s` is equivalent to `buffer = buffer + s`, but shorter and more convenient.

6. Now we use a for loop to simulate that we receive individual bytes one after the other, convert each of them into text, and append it to a string.

   Enter the following:

```
buffer = ""
for ch in [b'A', b'\n', b'X', b' ', b'1', b'\n']:
    ch = bytes.decode(ch)
    buffer += ch
print(buffer)
```

   This is not quite what we will need later – but it is heading in the right direction.

   Here is what happens:
   - `buffer` is initially an empty string.
   - In each loop iteration, a byte (`ch`) is converted into a string.
   - This string is appended to `buffer`.
   - At the end, `buffer` is output once.

   Note: Although there is only one `print` statement, two lines are displayed.
   This is because `buffer` contains two line breaks (`\n`) – and `print(buffer)` outputs the entire text including the line breaks it contains.

© Michael Lehn | CC BY-SA 4.0

7. Now the line break (`\n`) should no longer be appended to `buffer`. Instead, in this case the current content of `buffer` should be output, and `buffer` should then be reset to an empty string. To do this, change the code as follows:

```
buffer = ""
for ch in [b'A', b'\n', b'X', b' ', b'1', b'\n']:
    ch = bytes.decode(ch)
    if ch == '\n':
        print(f"Zeile> ‘{buffer}’")
        buffer = ""
    else:
        buffer += ch
```

The statement `print(f"Zeile> {buffer}")` produces formatted output: the placeholder `{buffer}` is automatically replaced by the current content of the variable `buffer`.

```
Zeile> ‘A’
Zeile> ‘X 1’
```

8. One more little thing: the microcontroller does not send line breaks as a single `b'\n'`, as is usual in the Unix world, but in a format that the Windows world also understands: a `b'\r'` (carriage return) followed by `b'\n'` (line feed).

   This is a perfect opportunity, though, to introduce an `elif` and the keyword `pass` – so that we simply ignore the character `\r`, as it should be. To do this, change the code as follows:

```
buffer = ""
for ch in [b'A', b'\r', b'\n', b'X', b' ', b'1', b'\r', b'\n']:
    ch = bytes.decode(ch)
    if ch == '\n':
        print(f"Zeile> {buffer}")
        buffer = ""
    elif ch == '\r':
        pass
    else:
        buffer += ch
```

   The `elif` condition detects the carriage return character (`'\r'`) and then skips it with `pass`. Everything else is appended to `buffer` as before.
   Additional task: Test what happens without the `elif` fix.

   `pass` in Python simply means: do nothing. This is comparable to a `nop` instruction (“no operation”) in assembly language – a placeholder that explicitly does nothing but has to be there syntactically.

9. **Now comes the grand finale**
   Now it is your turn to put the **knowledge from your experiments** into practice!

   - **Exit the Python shell** with `Control-D`.
   - Modify `serial-reader.py` so that the received bytes are output line by line as text. As soon as a complete line has been received (ending with `\n`), it should be output in full. The carriage return character (`\r`) is ignored – as before.
   - **Test your program** in the right terminal.
   - If everything works:
     `git commit -a -m "serial reader with line buffer"`
     `git push`
     With this, you have built your own serial non-blocking buffered line reader – congratulations!

© Michael Lehn | CC BY-SA 4.0

Goal of the following steps: Code organization is everything!

In the next steps, the existing code is reorganized so that it becomes modular, reusable, and easy to extend – for example, as a building block for later projects (such as our game). Along the way, we introduce the concept of object-oriented programming (OOP) step by step – each time with a concrete example.
In the end, two central functions (or methods) should be available:

- `connect(port)`
  Establishes the connection to the serial interface and returns an open Serial object, which is stored internally.
- `readline()`
  Checks whether a complete line has been received. Returns the line as a string if there is one, otherwise `None`. The received bytes are buffered internally and only output as a line once a line break (`\n`) has been received. The carriage return character (`\r`) is ignored.

The restructuring takes place in several stages:

1. **Modularization with global variables:**
   The logic is moved into two functions that use the global variables `ser` and `buffer`. This results in a first, easy-to-understand modular structure.
2. **Conversion into a class with static attributes and methods:**
   In the second step, the existing structure is turned into a class. `ser` and `buffer` become static attributes, `connect` and `readline` become static methods. This does not change how the code is used, but it brings order into the program.
3. **(Optional) Instance-based design for multiple use:**
   In a further step, the class could be extended so that several serial connections are possible at the same time – using instance attributes and methods.

No need to fear technical terms:
Do not let words such as *static attributes*, *instance attributes*, *methods*, *modules*, or *classes* put you off. We develop all of this step by step using concrete examples – and you will see: behind these "fancy" expressions are simple, easy-to-understand concepts.

© Michael Lehn | CC BY-SA 4.0

Step 3: Functions with shared variables

Fortunately, we already have a working program that reads the serial interface line by line. This already gives us a good idea of what the two planned functions should look like:

- The function `connect` uses `ser = serial.Serial(...)` to establish a connection to the USB port.
- The variable `ser` is then used in the function `readline` to
  - query whether new bytes are available (`ser.in_waiting > 0`), and
  - read a byte if needed (`ser.read(1)`).
- The function `readline` also uses a variable `buffer` to which new bytes are appended (after decoding). This buffer should have been initialized as an empty string before the first call of `readline` – this can conveniently be done in `connect`.

So we need two variables – `ser` and `buffer` – that can be used by both functions. The obvious solution: **global variables**.

However, Python has a nasty little catch when you want to **modify** global variables. Let us look at this using a simple proof of concept:
In the left terminal, create the file `controller.py` in an editor of your choice, with this content:

```
 1 ser = None
 2 buffer = None
 3
 4 def connect(port):
 5
 6     ser = f"connect to {port}"
 7     buffer = "empty"
 8
 9 def readline():
10
11     print(f"reading from {ser}")
12     buffer += ", some data"
13     print(f"buffer: {buffer}")
14
15 connect("/dev/ttyUSB0")
16 readline()
```

By “proof of concept” we mean: we use the simplest possible example to test whether two functions can share the same variables. The goal is **not yet** the serial interface – for now, the point is only that `connect` “prepares” something, and `readline` accesses it and carries on.

However, this does **not** work yet. We will **analyze and fix** this step by step in a moment, and then gradually build in the desired functionality.

Tasks

1. In the right terminal, run the program with `python controller.py` and confirm that there is an error.
2. The error message says (in a more or less understandable way) that in **line 12** the variable `buffer` is accessed, but that it **does not exist**. This is actually surprising – after all, we did create `buffer` in **line 2**, outside the function!
   But: **by default, Python assumes** that only **local variables are modified** inside a function. So if you write `buffer += ...`, Python interprets

© Michael Lehn | CC BY-SA 4.0

this as an attempt to modify a local variable `buffer` – and that one has not been defined before. So we have to **tell Python explicitly** that the global variable `buffer` is meant. This is done by changing **line 10** to:

```
10    global buffer
```

Run the program again now. Confirm that there is still an error message – **but a different one**.

3. **The new error message** says that the expression `buffer += ...` in **line 12** is not allowed – because `buffer` has the value `None` at this point. When `connect` is called, a variable `buffer` is indeed set in **line 7**, **but not the global variable** that we created in **line 2**. Instead, Python simply created a **local variable** `buffer` in the function `connect` – just as it always does by default. The global variable `buffer` therefore remains **unchanged** at `None`.

   Solution: In the function `connect`, too, we have to say explicitly that we want to modify the global variable `buffer`. And by now you know how to do that 😉

4. **There is no longer an error message**, but the program does **not do what it is supposed to do**. The output is:

   ```
   reading from None
   buffer: empty, some data
   ```

   What we would expect, however:

   ```
   reading from connect to /dev/ttyUSB0
   buffer: empty, some data
   ```

   In the function `connect`, we did correctly modify the global `buffer` – but `ser` was also only initialized locally. As a result, the global variable `ser` remains `None`, and that is exactly what we see in the output of `readline`.
   Solution: The variable `ser`, too, must be explicitly declared as **global** in the function `connect`. This is done by writing **line 5** as follows:

   ```
   5     global buffer, ser
   ```

   The nastiest part of it all: In the function `readline`, we also access `ser` in **line 11** – without writing global `ser` there. **And yet it works!**

   Why? Because in `readline` we only **read** `ser`, but do **not modify** it. Python allows access to global variables **without a** `global` **declaration, as long as you only read them**. Only when you try to modify them (as with `buffer += ...`) do you need the `global`.
   This is a nasty but important distinction!
5. Even though it is **not technically necessary**, change **line 10** as follows:
   ```
   10    global buffer, ser
   ```
   For ordinary mortals, it is then immediately **clear** that this function accesses **global variables** – whether for reading or writing. This makes the code **easier to understand** and protects against errors if you later do change something about `ser` after all.
6. Now add the new file to the repository:
   ```
   git add controller.py
   git commit -a -m "proof of concept"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 4: Extending the proof of concept

We can now copy code from our program `serial-reader.py` into `controller.py` to obtain the **actual functionality**. This involves a bit of typing – but it is not particularly tricky.

Below you can see what the code might look like **after a first iteration**. First, a brief overview of **what has been changed or added**:

- **Lines 1–2:** The required modules are imported:
  - `serial`, to communicate with the serial interface,
  - `time`, to put the process to sleep from time to time.
- **Lines 7–11:** The function `connect` now actually establishes a connection to the serial interface and initializes `ser`.
- **Lines 13–27:** The function `readline` reads all available characters, appends them to `buffer`, and checks for the end of a line:
  - If a **line feed** (`\n`) is found, it prints the content of `buffer` and resets `buffer`.
  - If a **carriage return** (`\r`) is found, nothing happens.
  - Otherwise, the character is added to `buffer`.
  - Important: The function **always** returns `None` – even if it succeeds! (We will fix that later.)
- **Lines 28–33:** The main program now uses `connect` and `readline`:
  - In **line 28**, the connection is established.
  - Then `readline` is called in an infinite loop.
  - If `readline` returns a line (that is, not `None`), it is output.

This already looks quite usable – but: it does not quite work yet. Because, as mentioned above, `readline` always returns `None`, even if a line has actually been read.

```
 1 import serial
 2 import time
 3
 4 ser = None
 5 buffer = None
 6
 7 def connect(port):
 8     global ser, buffer
 9     ser = serial.Serial(port, 9600)
10     buffer = ""
11     print(f"connected to {port}")
12
13 def readline():
14     global ser, buffer
15
16     while ser.in_waiting > 0:
17         ch = ser.read(1)
18         ch = bytes.decode(ch)
19         if ch == '\n':
20             print(f"Zeile> {buffer}")
21             buffer = ""
22         elif ch == '\r':
23             pass
24         else:
25             buffer += ch
26     return None
27
28 connect("/dev/ttyUSB0")
29 while 1:
30     line = readline()
31     if line is not None:
32         print(f"readline {line}")
33     time.sleep(0.1)
```

© Michael Lehn | CC BY-SA 4.0

Tasks

1. Adapt `controller.py` accordingly. Confirm that the received data is not output by the `print` in **line 32** (as actually intended), but by the output in **line 20**.

2. The obvious idea is to return the content of `buffer` in **line 20** instead of just outputting it. So change the line to `return buffer`. Confirm that the behavior has now changed, but is not yet quite what we want.
3. **Idea for solving the current problem**
   When we detect a complete line in `readline()` (that is, at the `\n`), then we:
   (I) store the current value of `buffer` in a new local variable, e.g., `line`
   (II) reset `buffer` to the empty string `""`
   (III) then return `line`

   Think for yourselves about how you can implement this strategy in Python code. Adapt the corresponding part of the function `readline()` accordingly. Then run the program and check whether everything works as desired.

4. **If everything now works as desired:**
   `git commit -a -m "connect, readline"`
   `git push`

Step 5: More structure – er, make it classy!

In our program, certain things **obviously belong together**:
- the two global variables `ser` and `buffer`, and
- the functions `connect` and `readline`.

We **know** this – at least **today**, because we worked hard to get these four loners to finally work together **as a team**.

But: if you do not yet know the code and how it works, you do **not immediately** see that these things belong together.

Fortunately, Python offers a language feature that lets you **make** exactly this structure **visible**. And yes – this is where the magic word comes up: `class`

Here is an almost self-explanatory example of how our code can be expressed **more clearly and in a more structured way** with a class. And yes – it is once again a **proof of concept**, which you will transfer to the real code in a moment:

```
 1 class Controller:
 2     ser = None
 3     buffer = None
 4
 5     def connect(port):
 6         Controller.ser = f"connected to {port}"
 7         Controller.buffer = "empty"
 8
 9     def readline():
10         print(f"reading from {Controller.ser}")
11         Controller.buffer += ", some data"
12         print(f"buffer: {Controller.buffer}")
13
14 Controller.connect("bar")
15 Controller.readline()
```

© Michael Lehn | CC BY-SA 4.0

Tasks

1. Write the example program above into a file named `controller_test.py` and run the program.

2. You should have received the following output:

   ```
   reading from connected to bar
   buffer: empty, some data
   ```

   And with that, you can **already guess** what this small program is meant to express:

   The variables `ser` and `buffer` as well as the functions `connect` and `readline` are part of the class `Controller`.

   One could also say:
   - They are encapsulated in the class,
   - or they belong to the namespace `Controller`.

   The world of object-oriented programming (OOP) may be full of clever terms – but what is meant is basically quite simple:
   - The variable `ser` in line 2 is actually called: `Controller.ser`
   - The variable `buffer` in line 3 is called: `Controller.buffer`
   - The function `connect` in line 5 is fully called: `Controller.connect`
   - And `readline` in line 9 is: `Controller.readline`

   👉 **And that is exactly why we no longer need a `global`!**

   Because with this language feature (`class`), it is **absolutely clear which variable** the code refers to – namely the one that **belongs to the class** `Controller`. This provides clarity, structure – and fewer surprises.

   So – now it should be easy to apply this principle to `controller.py`. And once again, it is your turn! 😎

3. If everything works as before (only now with class):
   `git commit -a -m "mit class"`
   `git push`

© Michael Lehn | CC BY-SA 4.0